

Principles of Computer Science II

Computational Thinking

Marco Zecchini

Sapienza University of Rome

Lecture 1

This is me!

Home page

Oct 01, 2024, 1 min read



Hi all, I am Marco Zecchini.

I am a Research Assistant at the **Department of Computer, Control and Management Engineering (DIAG)** of **Sapienza University of Rome**.

My research interests are Cryptography and Blockchain technology.



SAPIENZA
UNIVERSITÀ DI ROMA

Course catalogue

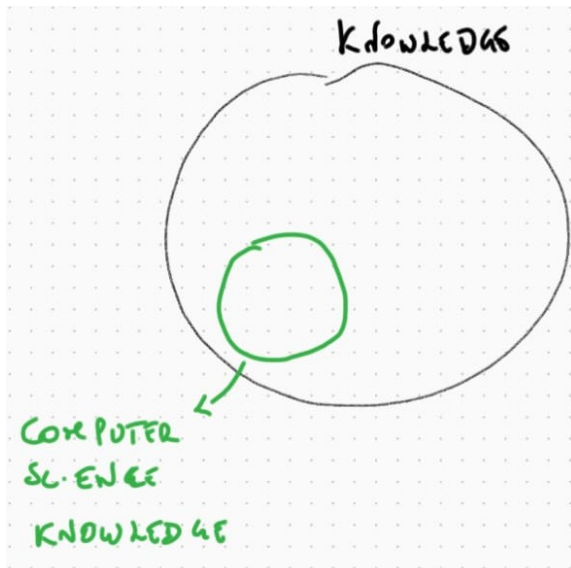
IT | EN

Curriculum(s) for 2026 - Bioinformatics (33455)

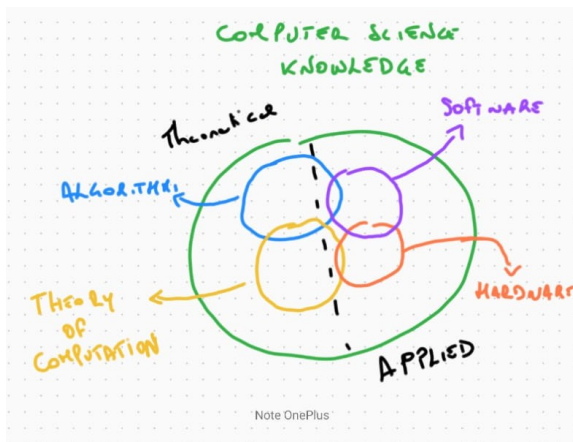
Single curriculum

Lesson [SSD] [Language]	Year	Semester	CFU
10630128 Principles of Mathematics [MATH-06/A] [ENG]	1st	1st	12
10630574 Organic and inorganic chemistry [CHEM-05/A, CHEM-03/A] [ENG]	1st	1st	12
10629560 Principles of Computer Science I [INFO-01/A] [ENG]	1st	1st	6
10631667 Principles of Physics [PHYS-03/A] [ENG]	1st	1st	6
10630574 Organic and inorganic chemistry [CHEM-05/A, CHEM-03/A] [ENG]	1st	2nd	12
10630575 Introduction to biomedical statistics [STAT-01/A, MEDS-24/A] [ENG]	1st	2nd	12
10632213 Biology of the cell [BIOS-10/A] [ENG]	1st	2nd	12
10629571 Genetics and computational genomics [BIOS-14/A] [ENG]	2nd	1st	6
10627584 Principles of Computer Science II [INF-05/A] [ENG]	2nd	1st	6
10628624 Microbiology [BIOS-15/A] [ENG]	2nd	1st	6

Do we really need this course?



Do we really need this course?



In PCSI

- Introduction to Programming and Computer Architecture;
- Introduction to Python. Installation and usage;
- Get and give information. Input/output from/to the user;
- Standard Python data structures: sequences, iterations, slices, lists, strings, dictionaries;
- Conditions. if, else, elif;
- Organizing the code. Functions, objects, classes, files;
- Python and multimedia. Data structures for images.

In PCSI

```
# Function to filter list items based on their length
def filter_items_by_length(items, min_length):
    # Filter items with length greater than or equal
    # to min_length
    return [item for item in items if len(item) >=
            min_length]

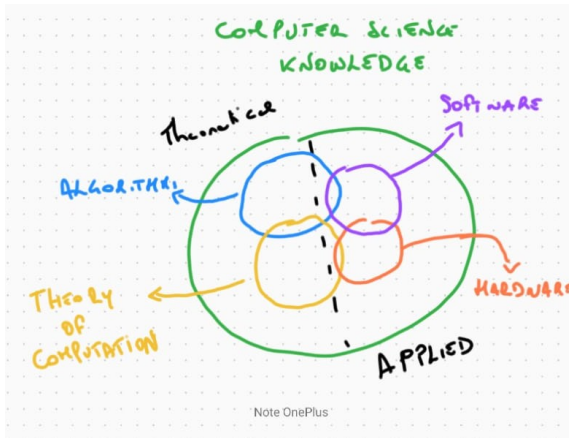
# Create a list of items
lst_listings = ['Item_1', 'Longer_Item_2', 'Item_3', '
                Very_Long_Item_4', 'Item_5']

min_length = 10

filtered_items = filter_items_by_length(lst_listings,
                                         min_length)

for item in filtered_items:
    print(item)
```

In PCSI



What about algorithms?

- **Problem:** We want to sort a vector of elements
- **Instance of the problem:** We want to sort this specific vector:

8	3	5	4	6
---	---	---	---	---

What about algorithms?

- **Problem:** We want to sort a vector of elements
- **Instance of the problem:** We want to sort this specific vector:

8	3	5	4	6
---	---	---	---	---
- **Application in Bioinformatics:** Sequence Database Search: Sorted DNA sequences can be used for faster searching in large sequence databases, such as BLAST, which relies on efficient sequence comparison algorithms

First algorithm: Selection Sort

Sorting a vector: [8, 3, 5, 4, 6]

Alg. Key intuition: Always Look for the smallest next minimum.

- **Initial Vector:**

8	3	5	4	6
---	---	---	---	---

- **Step 1: Find the minimum and swap with the first element [3]**

3	8	5	4	6
---	---	---	---	---

- **Step 2: Find the next minimum [4] and swap with second element**

3	4	5	8	6
---	---	---	---	---

- **Step 3: Find the next minimum [5] (already in place)**

3	4	5	8	6
---	---	---	---	---

- **Step 4: Find the next minimum [6] and swap with 4th element**

3	4	5	6	8
---	---	---	---	---

First Example: Insertion Sort

Sorting a vector: [8, 3, 5, 4, 6]

Alg. Key intuition: Divide the problem into smaller problem.

- **Initial Vector:**

8	3	5	4	6
---	---	---	---	---

- **Step 1: Insert 3 into the sorted portion [8]**

3	8	5	4	6
---	---	---	---	---

- **Step 2: Insert 5 into the sorted portion [3, 8]**

3	5	8	4	6
---	---	---	---	---

- **Step 3: Insert 4 into the sorted portion [3, 5, 8]**

3	4	5	8	6
---	---	---	---	---

- **Step 4: Insert 6 into the sorted portion [3, 4, 5, 8]**

3	4	5	6	8
---	---	---	---	---

Comparison of the two approaches

- **Insertion Sort:**

- Builds the sorted list one element at a time by inserting each new element into its correct position.
- Performs fewer comparisons when the list is nearly sorted.
- Easier to implement and understand for small lists.

- **Selection Sort:**

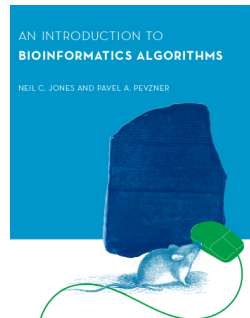
- Repeatedly selects the smallest (or largest) element from the unsorted part and swaps it with the first unsorted element.
- Always performs the same number of comparisons, regardless of how sorted the list is.
- Simpler in terms of swapping, but may involve more overall data movement.

- **Key Differences:**

- **Insertion Sort** tends to be faster when the list is almost sorted.
- **Selection Sort** always performs the same number of comparisons, but it swaps elements more frequently.

Other Example: the Change Problem

Jones, Pevzner: An Introduction to Bioinformatics Algorithms. MIT Press, 2004



Section 2.3, the US Change Problem: an everyday situation that hides a computational problem.

Computational problems are everywhere



You buy a pen for \$4.23 and pay with a \$5 bill: the cashier owes you 77 cents.

The United States Change Problem

- **Input:** an amount of money M , in cents.
- **Output:** the fewest coins among quarters (25), dimes (10), nickels (5) and pennies (1) whose values add up to M .

77 pennies are a correct answer, but not the one you want: correct is not enough, we want the **best** answer.

The cashier's algorithm

USCHANGE(M)

- 1 **while** $M > 0$
- 2 $c \leftarrow$ Largest coin that is smaller than (or equal to) M
- 3 Give coin with denomination c to customer
- 4 $M \leftarrow M - c$

The cashier's algorithm

USCHANGE(M)

```
1  while  $M > 0$ 
2       $c \leftarrow$  Largest coin that is smaller than (or equal to)  $M$ 
3      Give coin with denomination  $c$  to customer
4       $M \leftarrow M - c$ 
```

- For $M = 77$: quarter (52 left), quarter (27), quarter (2), penny (1), penny (0). Five coins.

Is that all?

- The rule is “always take the largest coin”. It gives the fewest coins for US coins. . .
- . . . but not for every coin system: with coins 25, 20, 10, 5, 1 and $M = 40$ it returns $25 + 10 + 5$, three coins, while $20 + 20$ is better.
- A **technique** that works in general is harder than it looks.

Later in the course we will formalize the problem and solve it correctly with dynamic programming.

So, why do need this course?

We need this course to learn how to recognize different computer science/bioinformatics problems, **solving** them with an **efficient** algorithms and, finally, **implementing** these algorithms in practice.

Do we really need this course?



SAPIENZA
UNIVERSITÀ DI ROMA

Course catalogue

IT | EN

Curriculum(s) for 2026 - Bioinformatics (33455)

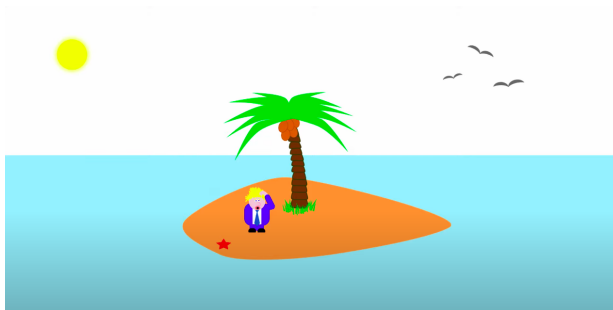
Single curriculum



Lesson [SSD] [Language]	Year	Semester	CFU
☰ ∨ 10630128 Principles of Mathematics [MATH-06/A] [ENG]	1st	1st	12
☰ ∨ 10630574 Organic and inorganic chemistry [CHEM-05/A, CHEM-03/A] [ENG]	1st	1st	12
∨ 10629560 Principles of Computer Science I [INFO-01/A] [ENG]	1st	1st	6
10631667 Principles of Physics [PHYS-03/A] [ENG]	1st	1st	6
☰ ∨ 10630574 Organic and inorganic chemistry [CHEM-05/A, CHEM-03/A] [ENG]	1st	2nd	12
☰ ∨ 10630575 Introduction to biomedical statistics [STAT-01/A, MEDS-24/A] [ENG]	1st	2nd	12
∨ 10632213 Biology of the cell [BIOS-10/A] [ENG]	1st	2nd	12
∨ 10629571 Genetics and computational genomics [BIOS-14/A] [ENG]	2nd	1st	6
∨ 10627584 Principles of Computer Science II [IINF-05/A] [ENG]	2nd	1st	6
∨ 10628624 Microbiology [BIOS-15/A] [ENG]	2nd	1st	6

Yes!

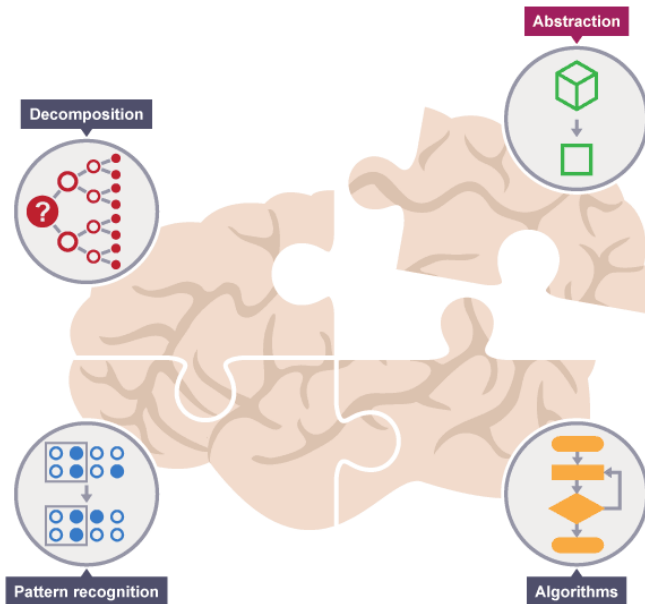
Computational Thinking



<https://www.youtube.com/watch?v=dHWmnayy8MY>

Computational Thinking

- Computational Thinking allows us to understand what needs to be solved.
- Four key techniques (cornerstones) to computational thinking:
 - ➊ Decomposition – breaking down a complex problem or system into smaller, more manageable parts
 - ➋ Pattern Recognition – looking for similarities among and within problems
 - ➌ Abstraction – focusing on the important information only, ignoring irrelevant detail
 - ➍ Algorithms through Logical Reasoning – developing a step-by-step solution to the problem, or the rules to follow to solve the problem



Computational Thinking vs Programming

Thinking computationally is not programming.

- ...not even thinking as a computer.
- Programming tells computer what to do / how to do it.
- Computational thinking enables us to understand what we need to tell to computers.
- ...what to program.

Examples:

- Explain to a friend how to drive to your house
- Organize a party at the park
- Prepare your luggage
- Teach a kid addition/subtraction
- ...

Decomposition

Turn a complex problem into one we can easily understand.

- ... probably you already do every day.
- The smaller parts are easier to solve.
- ... we already know/have the solutions.

Examples:

- Brushing our teeth
Which brush? How long? How hard? What toothpaste?
- Solving a crime
What crime? When? Where? Evidence? Witnesses? Recent similar crimes?
- ...

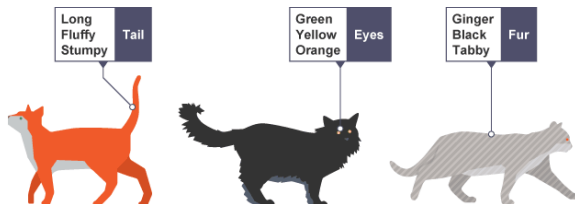
Pattern Recognition

We often find patterns among the smaller problems we examine.

- The patterns are similarities or characteristics that some of the problems share.

Example: Cats

- All cats share common characteristics. they all have eyes, tails and fur.
- Once we know how to describe one cat we can describe others, simply by following this pattern.



Abstraction

Hiding irrelevant details to focus on the essential features needed to understand and use a thing

- A compression process – multiple different pieces of constituent data to a single piece of abstract data.
e.g., “cat”
- Ambiguity – multiple different references.
e.g., “happiness”, “architecture”
- Simplification – no loss of generality
e.g., “red” - many different things can be red

Thought process wherein ideas are distanced from objects

Abstraction Example: Car vs Car Breaks



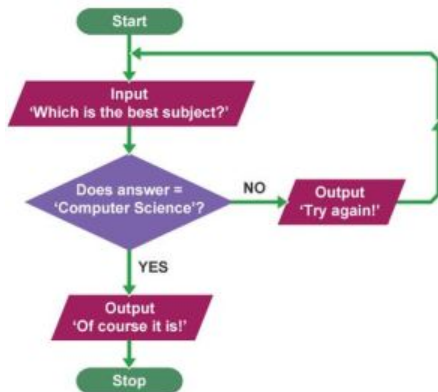
- Do we know how car breaks work?
- Do we know how to use them?

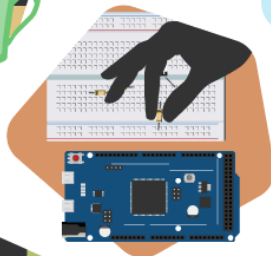
Filter out (ignore) the characteristics that we don't need in order to concentrate on those that we do.

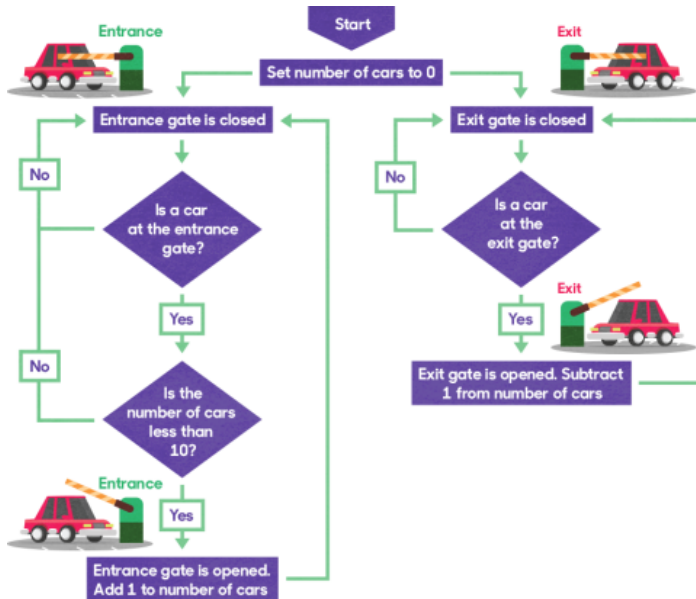
Algorithms

A plan, a set of step-by-step instructions to solve a problem.

- In an algorithm, each instruction is identified and the order in which they should be carried out is planned.







Bioinformatician's skill set

Biology & Medicine

- Basics in molecular and cell biology
- Measurement techniques

Computer Science

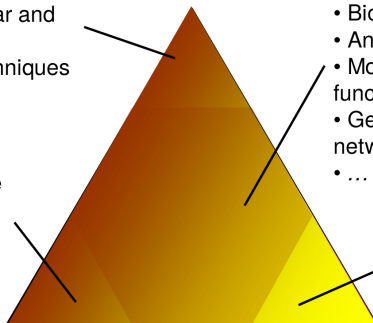
- Programming
- Databases
- Algorithmics

Bioinformatics

- Biological sequence analysis
- Biological databases
- Analysis of gene expression
- Modeling protein structure and function
- Gene, protein and metabolic networks
- ...

Mathematics and statistics

- Calculus
- Probability calculus
- Linear algebra



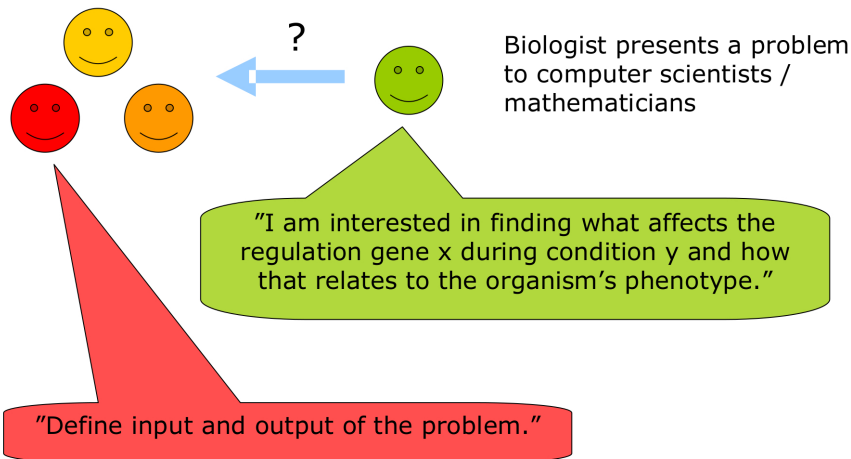
Where would you be in this triangle?

Prof. Juho Rousu, 2006 and Prof. I. Chatzigiannakis (for me!)

Bioinformatician's skill set

- Statistics, data analysis methods
 - Lots of data
 - High noise levels, missing values
 - #attributes $\gg$ #data points
- Programming languages
 - Scripting languages: **Python**, Perl, Ruby, ...
 - Extensive use of text file formats: need parsers
 - Integration of both data and tools
- Data structures, databases
 - New measurement techniques produce huge quantities of biological data.
- Scientific computation packages
 - R, Matlab/Octave, ...

Bioinformatician's Competences



Prof. Esa Pitkänen, 2008

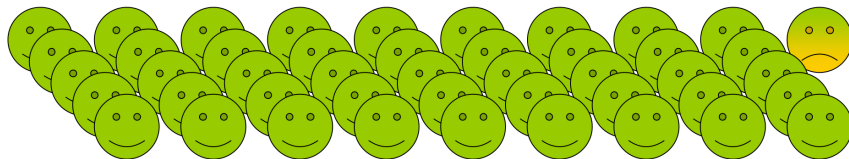
Bioinformatician's Competences

Bioinformatician is a part
of a group that consists
mostly of biologists.



Prof. Esa Pitkänen, 2008

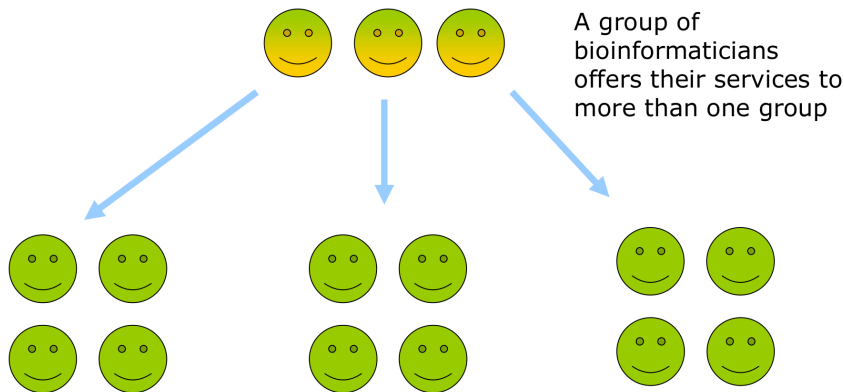
Bioinformatician's Competences



...biologist/bioinformatician ratio is important!

Prof. Esa Pitkänen, 2008

Bioinformatician's Competences



Prof. Esa Pitkänen, 2008

Axis 1: Python

- 1 Integrated Development Environment
- 2 Data Structures
- 3 Data Sets
- 4 Data Formats (BLAST, ...)
- 5 Data Storage
- 6 Visualization
- 7 Coding with Large Language Models (LLMs)

Axis 1: LLMs and coding, our approach

- LLMs (ChatGPT, Copilot, Claude, ...) are a **support** for coding, not a replacement for understanding the code.
- Two skills we will train:
 - ① write a **clear prompt**: input, expected output, constraints, an example;
 - ② **analyse the answer**: is the code correct? does it solve *our* problem? can I explain it?

Axis 1: exercises with LLMs

Two kinds of exercises, both starting from code produced by an LLM:

- 1 **Find the error.** The code looks fine but is wrong: find the bug, explain why it happens, fix it and write the prompt that would have avoided it.
- 2 **Explain the code.** Describe what the code does, line by line, and check it against what we asked for.

We will practise them in the hands-on lectures.

Example: find the error

Prompt: *"Write a Python function that computes the GC content of a DNA sequence."*

```
def gc_content(dna):
    count = 0
    for base in dna:
        if base == 'G' or 'C':
            count += 1
    return count / len(dna)

print(gc_content("ATGC")) # we expect 0.5, but it
                           prints 1.0
```

What does it print? Why? How do you fix it? Which prompt would have avoided it?

Example: the solution

Why 1.0? Python reads `base == 'G'` or `'C'` as `(base == 'G')` or `('C')`: a non-empty string is always true, so every base is counted.

Example: the solution

Why 1.0? Python reads `base == 'G'` or `'C'` as `(base == 'G')` or `('C')`: a non-empty string is always true, so every base is counted.

Fix: compare base with both letters (or use base in "GC").

```
if base == 'G' or base == 'C':  
    count += 1
```

Example: the solution

Why 1.0? Python reads `base == 'G'` or `'C'` as `(base == 'G')` or `('C')`: a non-empty string is always true, so every base is counted.

Fix: compare base with both letters (or use `base in "GC"`).

```
if base == 'G' or base == 'C':
    count += 1
```

Better prompt: *"... and add a test on an input whose GC content I know, e.g. ATGC (expected 0.5)."* Running the test would have revealed the bug immediately.

Axis 2: Algorithms

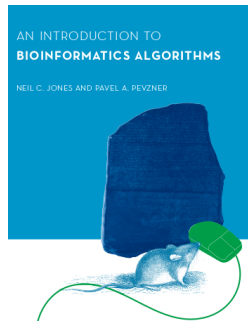
- 1 Complexity Analysis
- 2 Sorting
- 3 Exhaustive Search
- 4 Greedy Algorithms
- 5 Divide-and-Conquer Algorithms
- 6 Branch-and-Bound Algorithms
- 7 Data Mining Algorithms

Axis 3: Computing Infrastructures

- 1 Cloud Storage
- 2 Databases
- 3 Jupiter Notebook/Google Colab

Literature

Jones, Pevzner: An Introduction to
Bioinformatics Algorithms. MIT Press,
2004



JOHN M. ZELLE: Python Programming: An Introduction to
Computer Science (Third Edition)

Jeff Chang, Brad Chapman, Iddo Friedberg, Thomas Hamelryck,
Michiel de Hoon, Peter Cock, Tiago Antao, Eric Talevich, Bartek
Wilczyński: Biopython Tutorial and Cookbook

(Tentative) Lecture logistics

- Thursday, 2pm - 4pm: Theoretical lecture
- Monday, 4pm - 7pm: Hands-on lecture

Evaluation and exam

The course will be evaluated based on the performance of (a) written and practical implementation exam in class (we will do simulations), and (b) active participation of the student during the semester (students can gain bonus points during the semester).

Other information

- Website
- Google Classroom
- email: zecchini@diag.uniroma1.it